Bilkent University Electrical and Electronics Department

EE102-01 Lab 5 Report:

Seven-Segment Display

Muhammed Mesud Bilirgen – 22301965 – 24 March 2025

Purpose

The aim of this lab is to use seven segment displayer for multiple digits simultaneously. To do so, a clock divider is required to get different clock from internal BASYS3 clock for displaying digits simultaneously via 7-segment. Furthermore, the concept of persistence of vision is utilized.

Design Specification

The design is done in a modular fashion with the aim of clarity. The design has two modes which are multiplier and counter decided by “select_fcn” input. The top module is used the connect input and outputs of sub-modules to input and outputs of relative sub-module. Sub-modules are

- Multiplier: Computes an 8-bit product of x_i and y_i
- Counter: Increments periodically, reset by carry_i
- Display: Turns 16-bit number to values for cathodes and anodes using digit multiplexing
- Clock Divider: Reduces 100 MHz system clock to 1 Hz (for counter) and 1 kHz (for display refresh)

Inputs:

- x_i, y_i: 4-bit inputs for the multiplier (from switches)
- select_fcn: 3-bit input to select between multiplier and counter result (from switches)(“000” for multiplier or “001” for counter)
- carry_i: Used as a reset input for the counter and carry in for multiplier
- clock_100MHz: System clock from Basys3 board

Outputs:

- anodes_o (4 bits): Selects which digit is active on the 7-segment display
- cathodes_o (7 bits): Encodes which segments are lit for the active digit
- ultimate_o: Output flag (to a LED)(terminal count for counter or carry for multiplier)

Methodology

The design was implemented using hierarchical VHDL. The multiplier performs 4-bit by 4-bit binary multiplication which is done using n-bit adder sub-module. The counter is an 8-bit up-counter with selectable count limits. Considering the possibility of using this counter for further projects, the counter is made with counter limit inputs and terminal count output. The clock divider uses the 100 MHz input clock to generate two separate clocks: 1 Hz for the counter update and 1 kHz for 7-segment display multiplexing.

The seven segment display component selects one digit at a time using a 1 kHz clock and cycles through the 4 digits using the “anodes_o” signal. A decoder converts 4-bit hex digits to segment patterns.

All components were integrated into a top module. A 3-bit function selector (select_fcn) was used to choose between displaying the multiplier result and the counter value. The final design was tested both in simulation and on the Basys3 board.

Results

- The frequency of BASYS3 internal clock is 100 MHz.
- The 100 MHz clock can be divided by counting its rising edges and setting limits to increment another counter which is the functionality of clock divider (clockdivider.vhd in Appendices). Therefore, a slower clock signal can be created.
- The clock divider approach only allows frequencies that are integer divisions of the base clock (100 MHz ÷ N) so a clock with any arbitrary frequency lower than that of the internal clock may not be created.

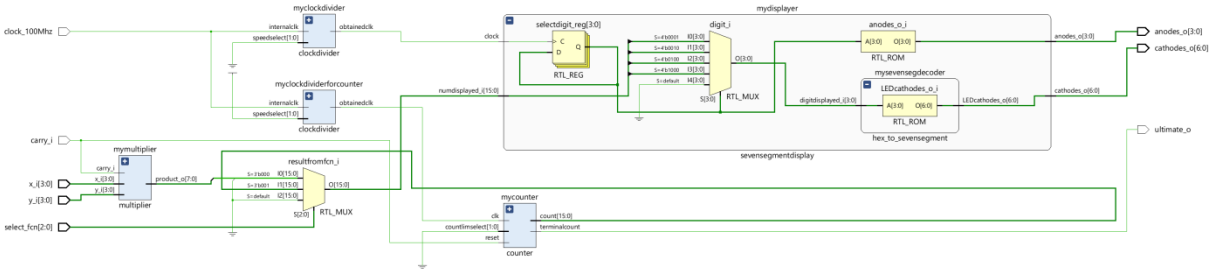

Figure 1: The RTL schematic

Some examples of functioning displayer in counter and multiplier mode as shown in examples Figure 2.1-8. The simulation waveform (Figure 2.0) shows how the seven segment displayer outputs anodes and cathodes according to inputted number as shown in the example Figure 2.5.

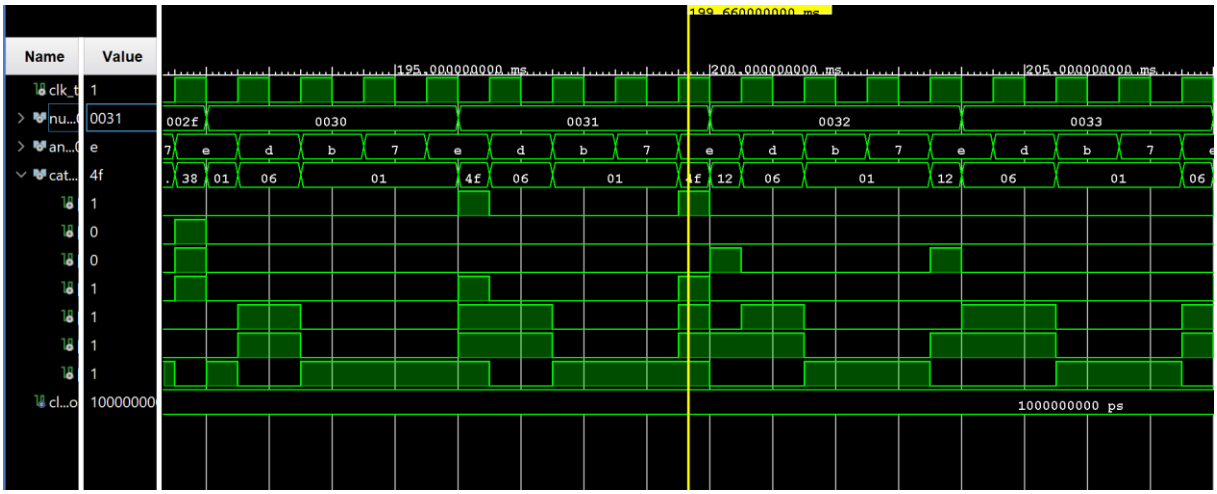

Figure 2.0: The Waveform

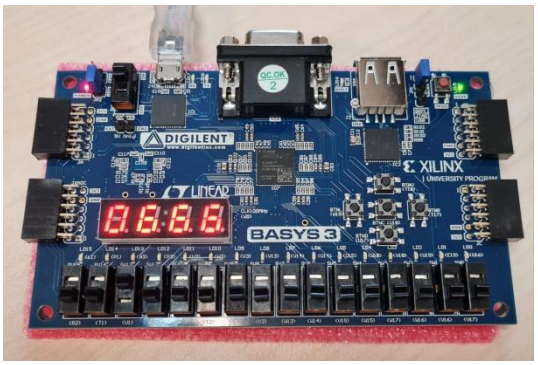

Figure 2.1: “0666” from counter

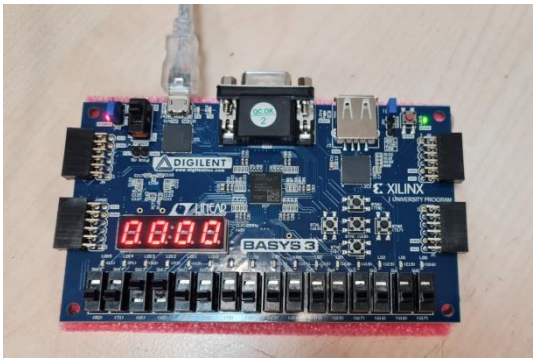

Figure 2.2: Reset = 1, “0000” from counter

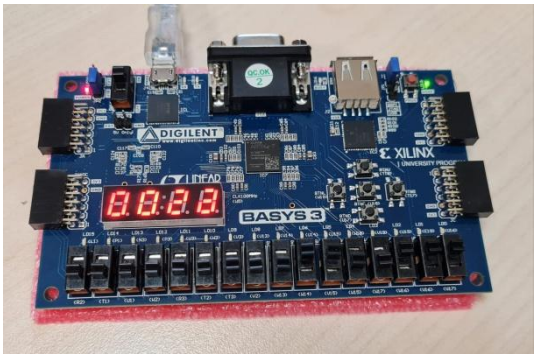

Figure 2.3: “0110” * ”0111” = “002A” from multiplier

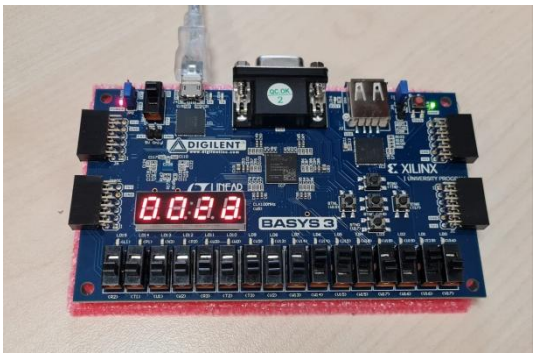

Figure 2.4: “0111” * “0110” = “002A”from multiplier

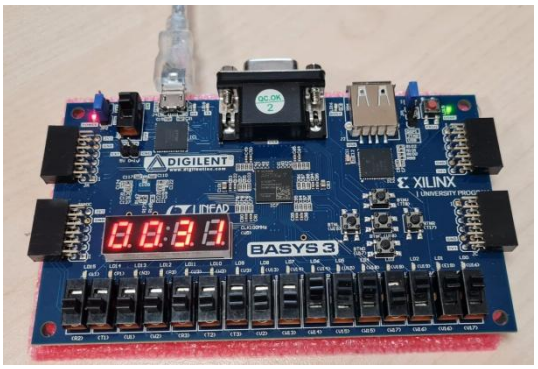

Figure 2.5: “0111” * ”0111” = “0031” from multiplier

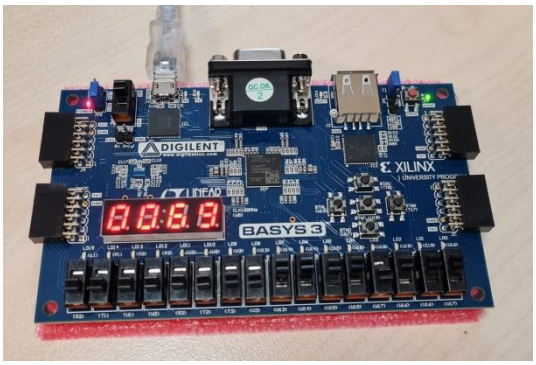

Figure 2.6: “1111” * ”0111” = “0069” from multiplier

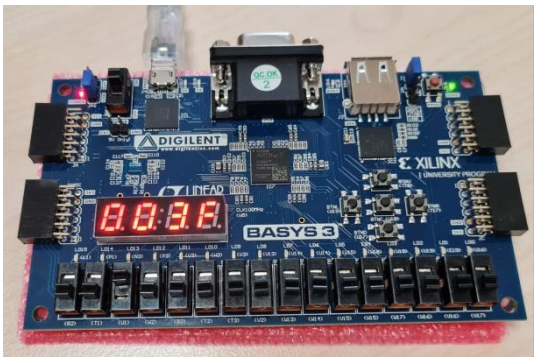

Figure 2.7: “003F” from counter

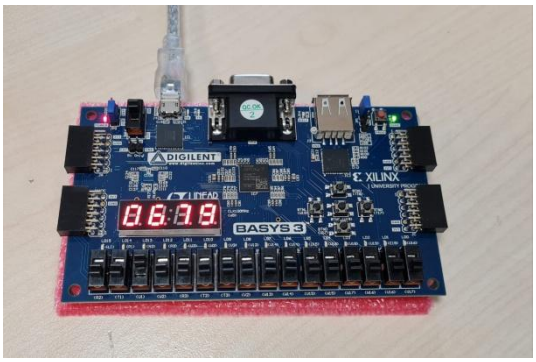

Figure 2.8: “0679” from counter

Conclusion

In this experiment, a modular digital system was successfully implemented on the Basys3 FPGA board using VHDL. The system included a multiplier, a configurable counter, a clock divider, and a 7-segment display controller. The modules were integrated into a top-level design. It allowed users to select between displaying the multiplier result and the counter value on a 4-digit 7-segment display. The design operated as expected in both simulation and on hardware. Simulation waveforms confirmed that the anode cycling and segment outputs matched the expected patterns for each digit.

My approach of using modules for everything and constituting unnecessary inputs and outputs might appear to be complicated way. In fact, by providing reusability and mobility, it eases designing. Overall, the objectives of the lab were met, and the implementation aligned with the intended design specifications.

Appendices

top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity top is Port (
clock_100Mhz: in std_logic;
select_fcn: in std_logic_vector (2 downto 0) := "001";
x_i                    : in std_logic_vector (3 downto 0);
y_i                    : in std_logic_vector (3 downto 0);
carry_i      : in std_logic;
anodes_o     : out std_logic_vector(3 downto 0);
cathodes_o   : out std_logic_vector (6 downto 0);
ultimate_o: out std_logic
);
end top;

architecture Behavioral of top is

signal clock_1kHz : std_logic;
signal clock_1Hz : std_logic;

signal resultfromfcn: std_logic_vector (15 downto 0);
signal resulttoLED: std_logic_vector (15 downto 0);
signal multiplier_result : std_logic_vector(7 downto 0);
signal counter_result : std_logic_vector(15 downto 0);

component clockdivider is
   Port ( internalclk : in STD_LOGIC;
        speedselect : in STD_LOGIC_VECTOR (1 downto 0);
        obtainedclk : out STD_LOGIC);
end component;

component sevensegmentdisplay Port (
clock : in std_logic;
numdisplayed_i : in std_logic_vector(15 downto 0);
anodes_o     : out std_logic_vector(3 downto 0);
cathodes_o   : out std_logic_vector (6 downto 0));
end component;

component counter is
port (
clk                                    : in std_logic;
reset          : in std_logic;
```

```
count                          : out std_logic_vector (15 downto 0);
terminalcount     : out std_logic
);
end component;

component multiplier is
port (
x_i                            : in std_logic_vector (3 downto 0);
y_i                            : in std_logic_vector (3 downto 0);
carry_i       : in std_logic;
product_o : out std_logic_vector (7 downto 0);
carry_o      : out std_logic
);
end component;

begin

process(select_fcn, multiplier_result, counter_result) begin
   case select_fcn is
      when "000" => resultfromfcn <= "00000000" & multiplier_result; -- Display Multiplier Output
      when "001" => resultfromfcn <= counter_result; -- Display Counter Output
      when others => resultfromfcn <= (others => '0'); -- Default: Display Zero
end case; end process;
resulttoLED <= resultfromfcn;

mymultiplier: multiplier port map (
x_i             => x_i,
y_i  => y_i,
carry_i       => carry_i,
product_o => multiplier_result,
carry_o => open );

mycounter: counter port map(
clk => clock_1Hz,
reset => carry_i,
count  => counter_result,
terminalcount => ultimate_o
);

myclockdividerforcounter: clockdivider port map(
internalclk => clock_100Mhz,
speedselect => "11",
obtainedclk => clock_1Hz);

myclockdivider: clockdivider port map(
internalclk => clock_100Mhz,
speedselect => "00",
obtainedclk => clock_1khz);

mydisplayer:  sevensegmentdisplay port map(
clock => clock_1khz,
numdisplayed_i => resulttoLED,
anodes_o   => anodes_o,
cathodes_o  => cathodes_o );

end Behavioral;
```

clockdivider.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity clockdivider is
   Port ( internalclk : in STD_LOGIC;
         speedselect : in STD_LOGIC_VECTOR (1 downto 0);
         obtainedclk : out STD_LOGIC);
end clockdivider;

architecture Behavioral of clockdivider is
```

```
signal count : integer range 0 to 100_000_000 := 0;
signal divisionfactor : integer := 100000000;
signal clockwaveform : std_logic := '0';

begin

process(internalclk) begin
   if rising_edge(internalclk) then
      case speedselect is
         when "00" => divisionfactor <= 100000;  -- 1 kHz
         when "01" => divisionfactor <= 200000;  -- 500 Hz
         when "10" => divisionfactor <= 1000000;  -- 100 Hz
         when "11" => divisionfactor <= 100000000; -- 1 Hz
         when others => divisionfactor <= 100000000; -- Default case
end case; end if; end process;

process(internalclk) begin
if rising_edge(internalclk) then
   if count = divisionfactor / 2 then
   clockwaveform <= not clockwaveform; count <= 0;
   else count <= count + 1 ;
   end if;
end if; end process;

obtainedclk <= clockwaveform;

end Behavioral;
```

counter.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity counter is
generic (clockfreq      : integer := 100_000_000);
port (
clk                                          : in std_logic;
countlimselect            : in std_logic_vector (1 downto 0) :="00";
reset           : in std_logic;
count                              : out std_logic_vector (15 downto 0);
terminalcount     : out std_logic
);
end counter;

architecture Behavioral of counter is

constant count100lim                  : integer := 65535;
constant count60lim                   : integer := 255;
constant count16lim     : integer := 16;
constant count10lim     : integer := 10;
signal countlim                                                  : integer range 0 to 65535          := 65535;

signal counter_int                                  : std_logic_vector (15 downto 0) := (others => '0');
signal termcount          : std_logic := '0';

begin

process(countlimselect) begin
   case countlimselect is
      when "00" => countlim <= count100lim;
      when "01" => countlim <= count60lim;
      when "10" => countlim <= count16lim;
      when others => countlim <= count10lim;
end case;end process;

process (clk, reset) begin
if reset = '1' then
counter_int <= (others => '0'); termcount <= '0';
elsif rising_edge(clk) then
```

```
    if unsigned(counter_int) >= countlim - 1 then
        termcount <= '1';
        counter_int <= (others => '0');
    else termcount <= '0'; counter_int <= counter_int + 1;
    end if;

end if; end process;

count <= counter_int;
terminalcount <= termcount;

end Behavioral;
```

sevensegmentdisplay.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity sevensegmentdisplay is
Port (
clock : in std_logic;
numdisplayed_i : in std_logic_vector(15 downto 0);
anodes_o      : out std_logic_vector(3 downto 0);
cathodes_o    : out std_logic_vector (6 downto 0)
);
end sevensegmentdisplay;

architecture Behavioral of sevensegmentdisplay is

signal selectdigit : std_logic_vector (3 downto 0) := "0001";
signal digit : std_logic_vector (3 downto 0);

component hex_to_sevensegment is
    Port (
        digitdisplayed_i : in std_logic_vector (3 downto 0);
        LEDcathodes_o    : out std_logic_vector (6 downto 0));
end component;

begin

process(clock) begin
    if rising_edge(clock) then
        selectdigit <= selectdigit(2 downto 0) & selectdigit(3);
    end if;
end process;

process(selectdigit, numdisplayed_i) begin
        case selectdigit is
            when "0001" => digit <= numdisplayed_i(3 downto 0);   anodes_o <= "1110"; -- Rightmost
            when "0010" => digit <= numdisplayed_i(7 downto 4);   anodes_o <= "1101"; -- Second
            when "0100" => digit <= numdisplayed_i(11 downto 8);  anodes_o <= "1011"; -- Third
            when "1000" => digit <= numdisplayed_i(15 downto 12); anodes_o <= "0111"; -- Leftmost
            when others => digit <= "0000"; anodes_o <= "1111"; -- all off
        end case;
end process;

mysevensegdecoder: hex_to_sevensegment
    port map (
        digitdisplayed_i => digit,
        LEDcathodes_o    => cathodes_o);

end Behavioral;
```

hextosevensegment.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
```

```
entity hex_to_sevensegment is
Port (
digitdisplayed_i    : in std_logic_vector (3 downto 0);
LEDcathodes_o       : out std_logic_vector (6 downto 0) );
end hex_to_sevensegment;

architecture Behavioral of hex_to_sevensegment is

begin

process(digitdisplayed_i) begin

case digitdisplayed_i is
when "0000" => LEDcathodes_o <= "0000001"; --0
when "0001" => LEDcathodes_o <= "1001111"; --1
when "0010" => LEDcathodes_o <= "0010010"; --2
when "0011" => LEDcathodes_o <= "0000110"; --3
when "0100" => LEDcathodes_o <= "1001100"; --4
when "0101" => LEDcathodes_o <= "0100100"; --5
when "0110" => LEDcathodes_o <= "0100000"; --6
when "0111" => LEDcathodes_o <= "0001111"; --7
when "1000" => LEDcathodes_o <= "0000000"; --8
when "1001" => LEDcathodes_o <= "0000100"; --9
when "1010" => LEDcathodes_o <= "0000010"; --A
when "1011" => LEDcathodes_o <= "1100000"; --B
when "1100" => LEDcathodes_o <= "0110001"; --C
when "1101" => LEDcathodes_o <= "1000010"; --D
when "1110" => LEDcathodes_o <= "0110000"; --E
when "1111" => LEDcathodes_o <= "0111000"; --F
when others => LEDcathodes_o <= "1111111";

end case; end process;
end Behavioral;
```

multiplier.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity multiplier is
port (
x_i                         : in std_logic_vector (3 downto 0);
y_i                         : in std_logic_vector (3 downto 0);
carry_i      : in std_logic;
product_o : out std_logic_vector (7 downto 0);
carry_o      : out std_logic
);
end multiplier;

architecture Behavioral of multiplier is

component nbit_adder is
generic (
n              : integer := 4
);
port (
x_i                         : in std_logic_vector (N-1 downto 0);
y_i                         : in std_logic_vector (N-1 downto 0);
carry_i      : in std_logic;
sum_o        : out std_logic_vector (N-1 downto 0);
carry_o      : out std_logic;
signedoverflow_o : out std_logic
);
end component nbit_adder;

signal s1, s2, s3, t0, t1, t2, t3 : STD_LOGIC_VECTOR (7 downto 0);
signal y0, y1, y2, y3 : STD_LOGIC_VECTOR (3 downto 0);
signal p0, p1, p2, p3 : STD_LOGIC_VECTOR (3 downto 0);
signal c1, c2, c3, o1, o2, cc : std_logic;

begin
```

```
y0 <= (others => y_i(0));
y1 <= (others => y_i(1));
y2 <= (others => y_i(2));
y3 <= (others => y_i(3));

p0 <= x_i and y0;
p1 <= x_i and y1;
p2 <= x_i and y2;
p3 <= x_i and y3;

t0 <= "0000" & p0;
t1 <= "000" & p1 & "0";
t2 <= "00" & p2 & "00" ;
t3 <= "0" & p3 & "000" ;

my8bitadder1: nbit_adder
generic map( n => 8 )
port map(
x_i => t0 ,
y_i => t1 ,
carry_o => c1 ,
sum_o => s1 ,
carry_i => carry_i
);

my8bitadder2: nbit_adder
generic map( n => 8 )
port map(
x_i => t2 ,
y_i => t3 ,
carry_o => c2 , -- c2 will be 0 anyway
sum_o => s2 ,
carry_i => '0'
);

my8bitadder3: nbit_adder
generic map( n => 8 )
port map(
x_i => s1 ,
y_i => s2 ,
carry_o => c3 ,
sum_o => s3 ,
carry_i => c1 -- so all sum and carries added now
);

product_o <= s3;
carry_o     <= c3;

end Behavioral;
```

nbit_adder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nbit_adder is
generic (
n              : integer := 4
);
port (
x_i                          : in std_logic_vector (N-1 downto 0);
y_i                          : in std_logic_vector (N-1 downto 0);
carry_i      : in std_logic;
sum_o       : out std_logic_vector (N-1 downto 0);
carry_o     : out std_logic;
signedoverflow_o : out std_logic
);
end nbit_adder;

architecture Behavioral of nbit_adder is
```

```
component fulladder is
Port ( carry_i : in STD_LOGIC;
        x_i : in STD_LOGIC;
        y_i : in STD_LOGIC;
        sum_o : out STD_LOGIC;
        carry_o : out STD_LOGIC);
end component fulladder;

signal carry : std_logic_vector (N downto 0) := (others => '0');

begin

carry(0) <= carry_i;

myfulladdergenerator: for i in 0 to N-1 generate
            fulladder_k : fulladder
            port map(
            x_i         => x_i(i),
            y_i         => y_i(i),
            carry_i => carry(i),
            sum_o       => sum_o(i),
            carry_o => carry(i+1)
            );
end generate;

carry_o     <= carry(N);
signedoverflow_o <= carry(n) XOR carry(n-1);

end Behavioral;
```

fulladder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fulladder is
   Port ( carry_i : in STD_LOGIC;
        x_i : in STD_LOGIC;
        y_i : in STD_LOGIC;
        sum_o : out STD_LOGIC;
        carry_o : out STD_LOGIC);
end fulladder;

architecture Behavioral of fulladder is

component halfadder is
port (
x_i         : in std_logic;
y_i         : in std_logic;
sum_o       : out std_logic;
carry_o     : out std_logic
);
end component halfadder;

signal sum1,c1,c2       : std_logic;

begin

myhalf1 : halfadder
port map(
x_i => x_i ,
y_i         => y_i ,
sum_o => sum1 ,
carry_o     => c1
);

myhalf2 : halfadder
port map(
x_i => sum1 ,
y_i         => carry_i ,
```

```
sum_o => sum_o ,
carry_o      => c2
);

carry_o <= c1 OR c2 ;

end Behavioral;
```

halfadder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity halfadder is
   Port ( x_i : in STD_LOGIC;
          y_i : in STD_LOGIC;
          sum_o : out STD_LOGIC;
          carry_o : out STD_LOGIC);
end halfadder;

architecture Behavioral of halfadder is

begin

sum_o <= x_i XOR y_i;
carry_o <= x_i AND y_i;

end Behavioral;
```

constraintlab5.xdc

```
# Clock signal
set_property PACKAGE_PIN W5 [get_ports clock_100Mhz]
set_property IOSTANDARD LVCMOS33 [get_ports clock_100Mhz]

#7 segment display
set_property PACKAGE_PIN W7 [get_ports {cathodes_o[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[6]}]
set_property PACKAGE_PIN W6 [get_ports {cathodes_o[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[5]}]
set_property PACKAGE_PIN U8 [get_ports {cathodes_o[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[4]}]
set_property PACKAGE_PIN V8 [get_ports {cathodes_o[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[3]}]
set_property PACKAGE_PIN U5 [get_ports {cathodes_o[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[2]}]
set_property PACKAGE_PIN V5 [get_ports {cathodes_o[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[1]}]
set_property PACKAGE_PIN U7 [get_ports {cathodes_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {cathodes_o[0]}]

set_property PACKAGE_PIN U2 [get_ports {anodes_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anodes_o[0]}]
set_property PACKAGE_PIN U4 [get_ports {anodes_o[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anodes_o[1]}]
set_property PACKAGE_PIN V4 [get_ports {anodes_o[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anodes_o[2]}]
set_property PACKAGE_PIN W4 [get_ports {anodes_o[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {anodes_o[3]}]


set_property PACKAGE_PIN V17 [get_ports {x_i[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {x_i[0]}]
set_property PACKAGE_PIN V16 [get_ports {x_i[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {x_i[1]}]
set_property PACKAGE_PIN W16 [get_ports {x_i[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {x_i[2]}]
set_property PACKAGE_PIN W17 [get_ports {x_i[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {x_i[3]}]

set_property PACKAGE_PIN W15 [get_ports {y_i[0]}]
```

```
		set_property IOSTANDARD LVCMOS33 [get_ports {y_i[0]}]
set_property PACKAGE_PIN V15 [get_ports {y_i[1]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {y_i[1]}]
set_property PACKAGE_PIN W14 [get_ports {y_i[2]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {y_i[2]}]
set_property PACKAGE_PIN W13 [get_ports {y_i[3]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {y_i[3]}]

set_property PACKAGE_PIN U1 [get_ports {select_fcn[0]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {select_fcn[0]}]
set_property PACKAGE_PIN T1 [get_ports {select_fcn[1]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {select_fcn[1]}]
set_property PACKAGE_PIN R2 [get_ports {select_fcn[2]}]
		set_property IOSTANDARD LVCMOS33 [get_ports {select_fcn[2]}]

set_property PACKAGE_PIN W2 [get_ports {carry_i}]
		set_property IOSTANDARD LVCMOS33 [get_ports {carry_i}]

# LEDs
set_property PACKAGE_PIN L1 [get_ports {ultimate_o}]
		set_property IOSTANDARD LVCMOS33 [get_ports {ultimate_o}]
```

lab5testbench.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity lab5testbench is
end lab5testbench;

architecture tb of lab5testbench is

   component sevensegmentdisplay is
      Port (
         clock         : in std_logic;
         numdisplayed_i : in std_logic_vector(15 downto 0);
         anodes_o       : out std_logic_vector(3 downto 0);
         cathodes_o     : out std_logic_vector(6 downto 0)
      );
   end component;

   signal clk_tb         : std_logic := '0';
   signal num_tb         : std_logic_vector(15 downto 0) := x"0000";
   signal anodes_tb      : std_logic_vector(3 downto 0);
   signal cathodes_tb    : std_logic_vector(6 downto 0);

   constant clk_period   : time := 1 ms;

begin

   uut: sevensegmentdisplay port map(
      clock         => clk_tb,
      numdisplayed_i => num_tb,
      anodes_o       => anodes_tb,
      cathodes_o     => cathodes_tb
   );

   clk_process: process
   begin
      while now < 500 ms loop  -- 100 values
         clk_tb <= '0';
         wait for clk_period / 2;
         clk_tb <= '1';
         wait for clk_period / 2;
      end loop;
      wait;
   end process;

   stim_proc: process
```

```
        variable count : integer := 0;
    begin
        while count <= 100 loop
            num_tb <= std_logic_vector(to_unsigned(count, 16));
            wait for 4 ms;
            count := count + 1;
        end loop;
        wait;
    end process;
end tb;
```